Searching and Sorting Algorithms Report

Nguyễn Dương Phúc

10/5/2025

Contents

1 Introduction

This report explains and analyzes common **searching** and **sorting** algorithms using simple examples.

We use the unsorted array:

$$[34, 7, 23, 32, 5, 62, 36, 14]$$

for sorting, and the sorted array:

$$[5, 7, 14, 23, 32, 34, 36, 62]$$

for searching.

2 Searching Algorithms

2.1 Linear Search

Idea: Check each element one by one until the target is found.
Steps (find 36):

- Step 1: Compare 5 → not target
- Step 2: Compare 7 → not target
- Step 3: Compare 14 → not target
- Step 4: Compare 23 → not target
- Step 5: Compare 32 → not target
- Step 6: Compare 34 → not target
- Step 7: Compare 36 → target found at index 6

Time complexity: O(n), space = O(1), best-case = O(1), worst-case = O(n).
Advantages: Works on sorted or unsorted arrays; simple; no extra memory needed.
Disadvantages: Slow for large arrays.
When to use: Small datasets, contiguous memory.

2.2 Binary Search

Idea: Works on sorted arrays. Pick middle element and reduce search space by half.
Steps (find 36):

- Step 1: left=0, right=7 → mid=(0+7)/2=3 → arr[3]=23 → 36>23 → left=4
- Step 2: left=4, right=7 → mid=(4+7)/2=5 → arr[5]=34 → 36>34 → left=6
- Step 3: left=6, right=7 → mid=(6+7)/2=6 → arr[6]=36 → target found

Time complexity: O(log n), space=O(1), best-case=O(1), worst-case=O(log n).
Advantages: Very fast for large sorted arrays; minimal memory.
Disadvantages: Requires sorted array; not suitable for unsorted data.
When to use: Large, sorted arrays.

2.3 Jump Search

Idea: Jump in blocks of size $\sqrt{n}$, then do linear search in that block.
Steps:

- Step 1: block size = 2 → jump arr[1]=7 → 36>7
- Step 2: jump arr[3]=23 → 36>23
- Step 3: jump arr[5]=34 → 36>34
- Step 4: jump arr[7]=62 → 36<62 → search block arr[6:7]

- Step 5: arr[6]=36 → target found

Time complexity: O($\sqrt{n}$), space=O(1).
Advantages: Faster than linear search for large sorted arrays; simple to implement.
Disadvantages: Requires sorted array; block size affects performance.
When to use: Large sorted arrays where binary search is not practical.

2.4 Ternary Search

Idea: Split array into 3 parts, compare with 2 midpoints.
Steps:

- Step 1: l=0, r=7 → mid1=2, mid2=5 → arr[2]=14, arr[5]=34 → 36>34 → search right third
- Step 2: l=6, r=7 → mid1=6, mid2=7 → arr[6]=36 → target found

Time complexity: O($\log_3$ n), space=O(1)
Advantages: Similar to binary search; theoretically reduces comparisons.
Disadvantages: Only works on sorted arrays; slightly more complex than binary search.
When to use: Sorted arrays; sometimes educational purposes.

2.5 Interpolation Search

Idea: Estimate position using formula: $pos = low + \frac{(x-arr[low])*(high-low)}{arr[high]-arr[low]}$
Steps: pos = 6 → arr[6]=36 → found
Time complexity: O(log log n) for uniform data, worst-case O(n)
Advantages: Very fast for uniformly distributed sorted data.
Disadvantages: Poor performance for non-uniform data; requires sorted array.
When to use: Sorted, uniformly distributed data.

2.6 Exponential Search

Idea: Check elements exponentially until overshoot, then binary search.
Steps:

- Step 1: i=0 → arr[0]=5 → 36>5
- Step 2: i=1 → arr[1]=7 → 36>7
- Step 3: i=2 → arr[2]=14 → 36>14
- Step 4: i=4 → arr[4]=32 → 36>32
- Step 5: i=8 → overshoot → binary search 4..7 → found at index 6

Time complexity: O(log n), space = O(1)
Advantages: Efficient for unbounded or infinite arrays.
Disadvantages: Requires sorted array; extra step of exponential search.
When to use: Large or infinite sorted datasets.

3 Sorting Algorithms

3.1 Selection Sort

Idea: Repeatedly select the smallest element and swap to front.
Steps:

- Step 1: find min=5 → swap with arr[0]=34 → [5,7,23,32,34,62,36,14]
- Step 2: min=7 → swap with arr[1]=7 → [5,7,23,32,34,62,36,14]
- Step 3: min=14 → swap with arr[2]=23 → [5,7,14,32,34,62,36,23]
- Step 4: min=23 → swap with arr[3]=32 → [5,7,14,23,34,62,36,32]
- Step 5: min=32 → swap with arr[4]=34 → [5,7,14,23,32,62,36,34]
- Step 6: min=34 → swap with arr[5]=62 → [5,7,14,23,32,34,36,62]
- Step 7: min=36 → swap with arr[6]=36 → [5,7,14,23,32,34,36,62]

Time complexity: $O(n^2)$, space=O(1)
Advantages: Simple; minimal moves; good if swaps are costly.
Disadvantages: Slow for large arrays; not stable.
When to use: Small arrays, low swap cost.

3.2 Insertion Sort

Idea: Build the sorted array one element at a time. Insert each element into its correct position in the sorted part.
Steps:

- Step 1: key=7 → move 34 → insert 7 → [7,34,...]
- Step 2: key=23 → move 34 → insert 23 → [7,23,34,...]
- Step 3: key=32 → move 34 → insert 32 → [7,23,32,34,...]
- Step 4: key=5 → move 7,23,32,34 → insert 5 → [5,7,23,32,34,...]
- Step 5: key=62 → insert at end → [5,7,23,32,34,62,...]
- Step 6: key=36 → move 62,34 → insert 36 → [5,7,23,32,34,36,62,...]
- Step 7: key=14 → move 23,32,34,36,62 → insert 14 → [5,7,14,23,32,34,36,62]

Time complexity: $O(n^2)$, space = O(1)
Advantages: Efficient for small datasets, adaptive (fast for nearly sorted arrays)
Disadvantages: Slow for large arrays
When to use: Small arrays or mostly sorted data

3.3 Bubble Sort

Idea: Repeatedly compare adjacent elements and swap if out of order. Large values "bubble" to the end.
Steps:

- Step 1: [34, 7, 23, 32, 5, 62, 36, 14] → swap 34 7 → [7, 34, 23, 32, 5, 62, 36, 14] ... continue
- Step 2: Repeat passes until no swaps → [5, 7, 14, 23, 32, 34, 36, 62]

Time complexity: O(n^2), space = O(1)
Advantages: Simple, stable
Disadvantages: Very slow for large arrays
When to use: Only for small datasets, educational purposes

3.4 Quick Sort

Idea: Choose a pivot, partition array into smaller/larger, sort recursively
Steps (pivot = middle element 32):

- Step 1: [34, 7, 23, 32, 5, 62, 36, 14] → partition → [7, 23, 5, 14, 32, 62, 36, 34]
- Step 2: Left [7, 23, 5, 14] pivot 23 → [7, 5, 14, 23]
- Step 3: Right [32, 62, 36, 34] pivot 36 → [32, 34, 36, 62]
- Step 4: Combine → [5, 7, 14, 23, 32, 34, 36, 62]

Time complexity: O(n log n) avg, O(n^2) worst, space = O(log n)
Advantages: Very fast on average, in-place
Disadvantages: Worst case O(n^2), not stable
When to use: Large arrays, general-purpose

3.5 Heap Sort

Idea: Build max heap, swap root with last element, heapify.
Steps (partial):

- Step 1: Build heap → [62, 34, 36, 32, 5, 23, 7, 14]
- Step 2: Swap 62 with last → [14, 34, 36, 32, 5, 23, 7, 62] → heapify → [36, 32, 14, 7, 5, 23, 34, 62]
- Step 3: Repeat → [5, 7, 14, 23, 32, 34, 36, 62]

Time complexity: O(n log n), space = O(1)
Advantages: In-place, guaranteed O(n log n)
Disadvantages: Not stable, more complex than simpler sorts
When to use: Large arrays when stability is not needed

3.6 Merge Sort

Idea: Divide array into halves, sort each, merge back.
Steps:

- Step 1: Divide [34,7,23,32] and [5,62,36,14]
- Step 2: Divide further → [34,7] [23,32] ... [5,62] [36,14]
- Step 3: Merge → [7,34,23,32] → [7,23,32,34]
- Step 4: Merge right → [5,62,36,14] → [5,14,36,62]
- Step 5: Merge both → [5,7,14,23,32,34,36,62]

Time complexity: O(n log n), space = O(n)
Advantages: Stable, predictable O(n log n)
Disadvantages: Requires extra memory
When to use: Large arrays, stable sorting needed

3.7 Radix Sort

Idea: Sort digits from least significant to most using counting sort.
Steps (sort [34, 7, 23, 32, 5, 62, 36, 14] by units → tens):

- Step 1: Units digit → [10, 5, 14, 32, 23, 36, 34, 62]
- Step 2: Tens digit → [5, 7, 14, 23, 32, 34, 36, 62]

Time complexity: O(n·k), space = O(n+k)
Advantages: Very fast for integers
Disadvantages: Works only for integers or fixed-length keys
When to use: Large numbers or integers, stable

3.8 Counting Sort

Idea: Count frequency of each number → reconstruct sorted array.
Steps:

- Step 1: Count occurrences → 5:1, 7:1, 14:1, 23:1, 32:1, 34:1, 36:1, 62:1
- Step 2: Rebuild → [5,7,14,23,32,34,36,62

Time complexity: O(n+k), space = O(k)
Advantages: Linear time for small range
Disadvantages: Uses extra memory for counts, only integers
When to use: Small range of integers

3.9 Bucket Sort

Idea: Divide numbers into buckets → sort each → merge.
Steps:

- Step 1: Distribute [34,7,23,32,5,62,36,14] into buckets (0-9,10-19,...)
- Step 2: Sort each bucket → [5,7,14,23,32,34,36,62]

Time complexity: O(n+k), space = O(n+k)
Advantages: Efficient for uniform distribution
Disadvantages: Performance depends on distribution, extra memory
When to use: Uniformly distributed data

3.10 Tim Sort

Idea: Hybrid of merge + insertion sort (used in Python/Java).
Steps:

- Step 1: Divide array into small runs (<=32 elements) → sort with insertion sort → [5,7,14,23,32,34,36,62]
- Step 2: Merge runs using merge sort → [5,7,14,23,32,34,36,62]

Time complexity: O(n log n), space = O(n)
Advantages: Stable, fast in practice, adaptive
Disadvantages: Slightly more complex implementation
When to use: General-purpose, standard library sort

4 Summary Table

Algorithm	**Type**	**Time Complexity**	**Main Concept**
Linear Search	Search	$O(n)$	Check each element
Binary Search	Search	$O(\log n)$	Divide by half
Jump Search	Search	$O(\sqrt{n})$	Jump in blocks
Ternary Search	Search	$O(\log_3 n)$	Divide into three
Interpolation Search	Search	$O(\log \log n)$	Predict position
Exponential Search	Search	$O(\log n)$	Expand by 2x
Selection Sort	Sort	$O(n^2)$	Pick smallest repeatedly
Insertion Sort	Sort	$O(n^2)$	Insert into sorted part
Bubble Sort	Sort	$O(n^2)$	Swap adjacent elements
Quick Sort	Sort	$O(n \log n)$	Divide by pivot
Merge Sort	Sort	$O(n \log n)$	Divide and merge
Heap Sort	Sort	$O(n \log n)$	Use max heap
Counting Sort	Sort	$O(n + k)$	Count frequency
Radix Sort	Sort	$O(n \cdot k)$	Sort by digits
Bucket Sort	Sort	$O(n + k)$	Group and merge
Tim Sort	Sort	$O(n \log n)$	Hybrid (Merge + Insertion)